a cyberwitchery talk

ai architecture katas



learning by building small models in plain python

veit heller
cyberwitchery lab

2026

section 01

01 why tiny models?

you (probably) have already used one

- a chatbot, a translator, code completion.
- under each is a neural network: arrays of parameters and matrix multiplication.
- you could build one.

frameworks are for production code

for learning, we will resort to minimalism.

the practice

build a small, runnable version yourself.

mostly python and numpy and small enough to read fully.

what you trade

+ visibility: every intermediate value is in front of you.
+ debuggability: a wrong value traces to one line.
+ intuition: the shapes show what each piece is for (puzzles, y'all!).

| out of scope: scale, throughput, benchmark numbers.

the setup

- just `numpy arrays`, emphasizing the shapes (they are most of the program).
- the notation in papers is shorthand for the array code.

a trip to the terminal to build a transformer ourselves.

section 02

02 a tiny transformer, end to end

what a transformer computes

tokens
→ embeddings
→ (attn + mlp) ×N
→ outputs

a stack of identical layers. each updates every position from all positions.

a token is a chunk of text, an embedding its vector. attention mixes across positions, the mlp transforms each position on its own.

self-attention

$$\text{scores} = Q\,K^{\top} / \sqrt{d_k}$$

$$\text{out} = \text{softmax}(\text{scores})\,V$$

a weighted average of all positions, with data-dependent weights.

Q, K, V are linear projections of the input. scores is (T, T): how much each position draws from the others.

softmax

```
def softmax(x):
    x = x - x.max(-1, keepdims=True)   # row max
    e = np.exp(x)
    return e / e.sum(-1, keepdims=True)
```

softmax maps a vector to a probability distribution. subtracting the row max is the numerically stable form: identical result, no overflow in exp.

residual connections

$$x = x + \text{attn}(\text{ln}(x))$$

$$x = x + \text{mlp}(\text{ln}(x))$$

the addition lets gradients reach every layer.

the residual connection, from resnets (2015). deep stacks need it to train.

layer normalization

$$y = \gamma \ (x - \mu) \ / \ \sqrt{\sigma^2 + \varepsilon} \ + \beta$$

keeps activations from drifting as depth grows.

μ, σ^2: mean and variance over the feature axis, per position. γ, β: learned. ε: avoids dividing by zero.

→ live

let’s build one.

a transformer block

~60 lines of numpy, no framework

attention, layer normalization, an mlp, residual connections, and the tests that pin them down. production models are this block, only wider and deeper. but: the operations are the same!

section 03

03 paper to prototype

read an equation as an array program

- every symbol is an array, every subscript is an axis.
- a $\sum_j$ is a matmul or a `.sum(axis=j)`.
- write the **shapes** in a comment before the code.

if the shapes line up, you usually understand the mechanics of the formula.

catch bugs early with boring things

- **shape asserts** at every boundary, not just the end.
- **unit tests for the math**: a known input, a hand-checked output.
- the **overfit test**: fit ten examples exactly. if you can't, it's broken.

section 04

04 multi-scale modelling (rgm)

structure across scales

many signals carry structure at several scales at once:

images, audio, text, physical fields.

model the coarse structure first, then refine it.

coarse to fine

$$\text{down: } (N) \rightarrow (N/2)$$
$$\text{up: } (N/2) \rightarrow (N)$$

a resolution pyramid: model coarse structure first, detail after.

coarse levels are small and global, fine levels carry details.

renormalization

$$x_\ell = \text{block}_\ell(\text{down}(x_{\ell+1}))$$

$$\text{for each level } \ell$$

coarse-grain repeatedly, keep the *same* model form at every scale.

borrowed from the renormalization group in physics: coarse-grain, model, repeat. each level learns its own parameters.

generativity

run the pyramid the other way: start coarse,
invent detail scale by scale.

a learned model at each scale adds the next level of detail. image example: it produces new images, it doesn't just process the ones you give it.

section 05

05 state-space models

attention has a cost problem

it compares every pair of positions, so doubling the input quadruples the work.

state-space models do sequence modelling in linear time: audio, dna, whole documents.

a state-space model

a loop with memory: walk the sequence, carry a running state, update it each step.

`state = A state + B input`, then `output = C state`. like an rnn, but every step is linear.

one model with two forms

because the loop is linear, it is also a single convolution over the sequence.

train as the convolution (whole sequence in parallel), run as the recurrence (one step, cheap memory). using the same weights!

attention also has a memory problem

to make the next token, it holds every earlier token in memory.

that store grows with the sequence and never shrinks.

constant memory

a fixed-size state summarises the whole past,

however long the sequence.

and it's durable: save or reload it. state is a first-class object.

section 06

06 your own tiny model lab

a reusable harness

we looked at three very different models.

we can use the same scaffold for each.

datasets, tests, math. build infra for your lab!

when to stop

the toy is for **understanding**.

when you need scale, reach for pytorch, jax, w/e.

the takeaway

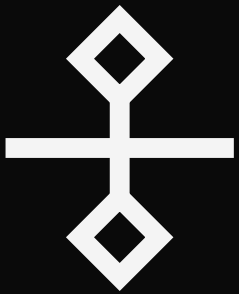
a paper is a specification.

implement the smallest version that runs.

try to stay true to the ideas only.

thanks.

questions?



veit heller veit@cyberwitchery.com

slides: github.com/cyberwitchery/talks